Mars 블록체인 개발자 가이드

목차

1. 프로젝트 개요
2. 프로젝트 구조 이해
3. 핵심 개념과 패턴
4. 개발 환경 설정
5. 커스텀 모듈 개발 가이드
6. 테스트 작성 가이드
7. IBC 통합 가이드
8. 성능 최적화
9. 보안 모범 사례
10. 배포 및 운영
11. 문제 해결

1. 프로젝트 개요

1.1 Mars 블록체인이란?

Mars는 Cosmos SDK를 기반으로 구축된 블록체인 플랫폼으로, 다음과 같은 특징을 가집니다:

- **모듈화된 아키텍처**: 각 기능이 독립적인 모듈로 구성
- **IBC 지원**: 다른 Cosmos 체인과의 상호 운용성

- **거버넌스 시스템**: 온체인 거버넌스를 통한 민주적 의사결정
- **확장 가능성**: 새로운 모듈 추가를 통한 기능 확장

1.2 기술 스택

```
┌──────────────────────────────────────┐
│ 애플리케이션 레이어 │
│ (Mars 커스텀 모듈 + SDK 모듈) │
├──────────────────────────────────────┤
│ Cosmos SDK │
│ (Auth, Bank, Staking, Gov...) │
├──────────────────────────────────────┤
│ IBC │
│ (Inter-Blockchain Communication) │
├──────────────────────────────────────┤
│ CometBFT │
│ (합의 엔진) │
├──────────────────────────────────────┤
│ 데이터베이스 │
│ (LevelDB, BadgerDB...) │
└──────────────────────────────────────┘
```

1.3 주요 구성 요소

- **앱 레이어** (`app/`): 애플리케이션 초기화 및 설정

- **커스텀 모듈** (`x/mars/`): Mars 체인의 고유 기능
- **CLI** (`cmd/marsd/`): 명령줄 인터페이스
- **프로토콜 정의** (`proto/`): gRPC 서비스 및 메시지 정의

2. 프로젝트 구조 이해

2.1 디렉토리 구조 상세 분석

```
mars/
├── app/ # 애플리케이션 핵심
| ├── app.go # 메인 앱 구조체 및 초기화
| ├── app_config.go # 모듈 설정 및 의존성 주입
| ├── config.go # 기본 설정 (주소 prefix 등)
| ├── export.go # 상태 내보내기
| ├── genesis.go # 제네시스 처리
| └── ibc.go # IBC 모듈 설정
├── cmd/marsd/ # CLI 애플리케이션
| ├── cmd/ # 명령어 정의
| └── main.go # 진입점
├── x/mars/ # Mars 커스텀 모듈
| ├── keeper/ # 비즈니스 로직
| ├── types/ # 타입 정의
| └── module/ # 모듈 등록
├── proto/mars/ # 프로토콜 정의
```

```
└── docs/ # API 문서
```

2.2 파일별 역할과 책임

app/app.go

역할: 애플리케이션의 중앙 허브

```go
type App struct {
*runtime.App // Cosmos SDK 런타임
// 코덱 시스템
legacyAmino *codec.LegacyAmino
appCodec codec.Codec
txConfig client.TxConfig
interfaceRegistry codectypes.InterfaceRegistry
// Keeper들 - 각 모듈의 비즈니스 로직 관리자
AuthKeeper authkeeper.AccountKeeper
BankKeeper bankkeeper.Keeper
StakingKeeper *stakingkeeper.Keeper
// ... 기타 keeper들
}
```

주요 기능:

- 모든 모듈의 Keeper 인스턴스 보유
- 의존성 주입을 통한 모듈 간 연결

- API 라우트 등록 및 서비스 제공

app/app_config.go

역할: 모듈 설정 및 실행 순서 정의

```
// 모듈 실행 순서 설정
BeginBlockers: []string{
minttypes.ModuleName, // 토큰 발행
distrtypes.ModuleName, // 보상 분배
slashingtypes.ModuleName, // 슬래싱 (분배 후 실행 필수)
// ...
}
```

중요한 순서 규칙:

- 토큰 발행이 분배보다 먼저 실행
- 슬래싱이 분배 후 실행 (CanWithdrawInvariant 유지)
- 스테이킹이 마지막에 검증자 세트 업데이트

2.3 빌드 시스템 이해

Makefile 주요 타겟들

테스트 관련:

```
test-unit: # 단위 테스트 실행
test-race: # 경쟁 상태 테스트
test-cover: # 커버리지 리포트 생성
```

```
bench: # 벤치마크 테스트
```

빌드 관련:

```
install: # 바이너리 빌드 및 설치
proto-gen: # Protocol Buffer 코드 생성
lint: # 코드 품질 검사
```

개발 도구:

```
govet: # Go 정적 분석
govulncheck: # 보안 취약점 검사
clean: # 캐시 및 임시 파일 정리
```

3. 핵심 개념과 패턴

3.1 Keeper 패턴

Keeper는 Cosmos SDK의 핵심 설계 패턴으로, 각 모듈의 상태 관리와 비즈니스 로직을 담당합니다.

```
// Keeper 구조체 예시
type Keeper struct {
storeService corestore.KVStoreService // 저장소 접근
cdc codec.Codec // 직렬화
addressCodec address.Codec // 주소 처리
authority []byte  // 권한 관리
```

```
// Collections 기반 상태 관리
Schema collections.Schema
Params collections.Item[types.Params]
}
```

Keeper의 책임:

- **상태 관리**: 모듈의 모든 상태 데이터 관리
- **비즈니스 로직**: 모듈의 핵심 기능 구현
- **권한 검증**: 요청자의 권한 확인
- **데이터 검증**: 입력 데이터의 유효성 검사

3.2 Collections 패턴

Cosmos SDK v0.50+에서 도입된 현대적인 상태 관리 방식입니다.

```
// 기존 방식 (레거시)
store := ctx.KVStore(k.storeKey)
bz := store.Get([]byte("params"))
var params types.Params
k.cdc.MustUnmarshal(bz, &params)

// Collections 방식 (현대적)
params, err := k.Params.Get(ctx)
if err != nil {
return err
```

```
}
```

Collections의 장점:

- **타입 안전성**: 컴파일 타임 타입 검증
- **자동 직렬화**: 코덱을 통한 자동 변환
- **구조화된 키**: 네임스페이스 기반 키 관리
- **성능 최적화**: 효율적인 저장소 접근

3.3 의존성 주입 패턴

Cosmos SDK v0.50+에서 도입된 의존성 주입 시스템을 사용합니다.

```
// 의존성 주입 설정
if err := depinject.Inject(appConfig,
&appBuilder,
&app.appCodec,
&app.AuthKeeper,
&app.BankKeeper,
// ... 기타 의존성들
); err != nil {
panic(err)
}
```

장점:

- **자동 의존성 해결**: 모듈 간 의존성 자동 주입

- **설정 중앙화**: app_config.go에서 모든 설정 관리
- **테스트 용이성**: 모의 객체 주입 가능

3.4 메시지 처리 패턴

gRPC 기반의 메시지 처리 시스템을 사용합니다.

```
// 메시지 서버 구현
type msgServer struct {
Keeper
}


func (k msgServer) UpdateParams(ctx context.Context, req *types.MsgUpdateParams)
// 1. 권한 검증
if !bytes.Equal(k.GetAuthority(), authority) {
return  nil, errors.New("unauthorized")
}
// 2. 유효성 검증
if err := req.Params.Validate(); err != nil {
return  nil, err
}
// 3. 상태 업데이트
if err := k.Params.Set(ctx, req.Params); err != nil {
return  nil, err
}
```

```
return &types.MsgUpdateParamsResponse{}, nil
}
```

4. 개발 환경 설정

4.1 필수 도구 설치

```
# Go 설치 (1.21+)
go version

# Ignite CLI 설치
curl https://get.ignite.com/cli@v28.5.0! | bash

# Protocol Buffers 컴파일러
# macOS
brew install protobuf

# Ubuntu
sudo apt install protobuf-compiler

# 추가 도구
go install github.com/cosmos/gogoproto/protoc-gen-gocosmos@latest
```

```
go install  github.com/grpc-ecosystem/grpc-gateway/protoc-gen-grpc-gateway@lates
```

4.2 프로젝트 클론 및 빌드

```
# 프로젝트 클론
git clone <mars-repo-url>
cd mars

# 의존성 설치
go mod  download

# 프로토콜 버퍼 코드 생성
make proto-gen

# 빌드
make install

# 개발 서버 시작
ignite chain  serve
```

4.3 개발 도구 설정

VS Code 설정 (.vscode/settings.json)

```
{
"go.toolsManagement.checkForUpdates": "local",
"go.useLanguageServer": true,
"go.lintTool": "golangci-lint",
"go.formatTool": "goimports",
"protoc": {
"options": [
"--proto_path=proto",
"--proto_path=third_party/proto"
]
}
}
```

Git Hooks 설정

```
# pre-commit 훅 설정
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/sh
make lint
make test
EOF
```

```
chmod +x .git/hooks/pre-commit
```

5. 커스텀 모듈 개발 가이드

5.1 새로운 모듈 생성

```
# Ignite CLI를 사용한 모듈 스캐폴딩
ignite scaffold module mymodule

# 메시지 추가
ignite scaffold message create-item name:string description:string --module

# 쿼리 추가
ignite scaffold query get-item id:uint --module mymodule

# 타입 추가
ignite scaffold type item name:string description:string creator:string --
```

5.2 Keeper 구현

```
// x/mymodule/keeper/keeper.go
package keeper
```

```go
import (
"cosmossdk.io/collections"
"cosmossdk.io/core/address"
corestore "cosmossdk.io/core/store"
"github.com/cosmos/cosmos-sdk/codec"
"mychain/x/mymodule/types"
)

type Keeper struct {
storeService corestore.KVStoreService
cdc codec.Codec
addressCodec address.Codec
authority []byte
// Collections 정의
Schema collections.Schema
Items collections.Map[uint64, types.Item]
NextId collections.Sequence
}

func NewKeeper(
storeService corestore.KVStoreService,
cdc codec.Codec,
addressCodec address.Codec,
```

```
authority []byte,

) Keeper {

sb := collections.NewSchemaBuilder(storeService)

k := Keeper{

storeService: storeService,

cdc: cdc,

addressCodec: addressCodec,

authority: authority,

Items: collections.NewMap(sb, types.ItemsKey, "items", collections.Uint64Key, co

NextId: collections.NewSequence(sb, types.NextIdKey, "next_id"),

}

schema, err := sb.Build()

if err != nil {

panic(err)

}

k.Schema = schema

return k

}
```

5.3 메시지 핸들러 구현

```
// x/mymodule/keeper/msg_server.go

package keeper
```

```go
import (
"context"
"mychain/x/mymodule/types"
)

type msgServer struct {
Keeper
}

func NewMsgServerImpl(keeper Keeper) types.MsgServer {
return &msgServer{Keeper: keeper}
}

func (k msgServer) CreateItem(ctx context.Context, req *types.MsgCreateItem) (*t
// 1. 입력 검증
if req.Name == "" {
return  nil, types.ErrInvalidName
}
// 2. 권한 검증 (필요한 경우)
creator, err := k.addressCodec.StringToBytes(req.Creator)
if err != nil {
return  nil, err
```

```
}
// 3. 비즈니스 로직 실행
id, err := k.NextId.Next(ctx)
if err != nil {
return  nil, err
}
item := types.Item{
Id: id,
Name: req.Name,
Description: req.Description,
Creator: req.Creator,
}
// 4. 상태 저장
if err := k.Items.Set(ctx, id, item); err != nil {
return  nil, err
}
// 5. 이벤트 발행
ctx.EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeCreateItem,
sdk.NewAttribute(types.AttributeKeyItemId, fmt.Sprintf("%d", id)),
sdk.NewAttribute(types.AttributeKeyCreator, req.Creator),
),
```

```
)
return &types.MsgCreateItemResponse{Id: id}, nil
}
```

5.4 쿼리 핸들러 구현

```
// x/mymodule/keeper/query.go
package keeper

import (
"context"
"google.golang.org/grpc/codes"
"google.golang.org/grpc/status"
"mychain/x/mymodule/types"
)

func (k Keeper) GetItem(ctx context.Context, req *types.QueryGetItemRequest) (*t
if req == nil {
return  nil, status.Error(codes.InvalidArgument, "invalid request")
}
item, err := k.Items.Get(ctx, req.Id)
if err != nil {
```

```go
if errors.Is(err, collections.ErrNotFound) {
return  nil, status.Error(codes.NotFound, "item not found")
}
return  nil, status.Error(codes.Internal, err.Error())
}
return &types.QueryGetItemResponse{Item: item}, nil
}


func (k Keeper) ListItems(ctx context.Context, req *types.QueryListItemsRequest)
if req == nil {
return  nil, status.Error(codes.InvalidArgument, "invalid request")
}
var items []types.Item
err := k.Items.Walk(ctx, nil, func(key uint64, value types.Item) (bool, error) {
items = append(items, value)
return  false, nil
})
if err != nil {
return  nil, status.Error(codes.Internal, err.Error())
}
return &types.QueryListItemsResponse{Items: items}, nil
}
```

5.5 모듈 등록

```go
// x/mymodule/module/module.go
package module

import (
"context"
"encoding/json"
"github.com/cosmos/cosmos-sdk/client"
"github.com/cosmos/cosmos-sdk/codec"
codectypes "github.com/cosmos/cosmos-sdk/codec/types"
sdk "github.com/cosmos/cosmos-sdk/types"
"github.com/cosmos/cosmos-sdk/types/module"
"mychain/x/mymodule/keeper"
"mychain/x/mymodule/types"
)

// AppModuleBasic 구현
type AppModuleBasic struct{}

func (AppModuleBasic) Name() string { return types.ModuleName }

func (AppModuleBasic) RegisterLegacyAminoCodec(cdc *codec.LegacyAmino) {
```

```
types.RegisterLegacyAminoCodec(cdc)
}

func (AppModuleBasic) RegisterInterfaces(registry codectypes.InterfaceRegistry)
types.RegisterInterfaces(registry)
}

// AppModule 구현
type AppModule struct {
AppModuleBasic
keeper keeper.Keeper
}

func NewAppModule(keeper keeper.Keeper) AppModule {
return AppModule{
AppModuleBasic: AppModuleBasic{},
keeper: keeper,
}
}

// 서비스 등록
func (am AppModule) RegisterServices(cfg module.Configurator) {
types.RegisterMsgServer(cfg.MsgServer(), keeper.NewMsgServerImpl(am.keeper))
```

```
types.RegisterQueryServer(cfg.QueryServer(), am.keeper)
}

// 제네시스 처리
func (am AppModule) InitGenesis(ctx sdk.Context, cdc codec.JSONCodec, data json.
var genesisState types.GenesisState
cdc.MustUnmarshalJSON(data, &genesisState)
// 제네시스 상태 초기화
for _, item := range genesisState.Items {
am.keeper.Items.Set(ctx, item.Id, item)
}
return []abci.ValidatorUpdate{}
}
```

5.6 커스텀 모듈 개발 모범 사례

5.6.1 상태 설계 원칙

1. Collections 우선 사용

```
// 권장: Collections 사용
Items: collections.Map[uint64, types.Item]

// 비권장: 직접 KVStore 사용
```

```
store.Set([]byte("item_1"), k.cdc.MustMarshal(&item))
```

2. 적절한 키 설계

```
// 좋은 예: 구조화된 키
var (
ItemsKey = collections.NewPrefix(0)
CountersKey = collections.NewPrefix(1)
IndexKey = collections.NewPrefix(2)
)

// 나쁜 예: 하드코딩된 문자열
store.Set([]byte("items"), data)
```

3. 인덱스 활용

```
// 효율적인 쿼리를 위한 인덱스 설계
ItemsByCreator: collections.NewIndexedMap(
sb, types.ItemsByCreatorKey, "items_by_creator",
collections.Uint64Key,
codec.CollValue[types.Item](cdc),
types.NewItemsIndex(sb),
)
```

5.6.2 에러 처리 패턴

1. 명확한 에러 정의

```
// x/mymodule/types/errors.go
var (
ErrInvalidName = sdkerrors.Register(ModuleName, 1100, "invalid name")
ErrItemNotFound = sdkerrors.Register(ModuleName, 1101, "item not found")
ErrUnauthorized = sdkerrors.Register(ModuleName, 1102, "unauthorized")
ErrInvalidDescription = sdkerrors.Register(ModuleName, 1103, "invalid descriptio
)
```

2. 계층적 에러 처리

```
func (k msgServer) CreateItem(ctx context.Context, req *types.MsgCreateItem) (*t
// 입력 검증
if err := k.validateCreateItemRequest(req); err != nil {
return  nil, err
}
// 비즈니스 로직
if err := k.executeCreateItem(ctx, req); err != nil {
return  nil, err
}
return &types.MsgCreateItemResponse{}, nil
}
```

5.6.3 이벤트 설계

1. 구조화된 이벤트

```
// 이벤트 타입 정의
const (
EventTypeCreateItem = "create_item"
EventTypeUpdateItem = "update_item"
EventTypeDeleteItem = "delete_item"
)

// 속성 키 정의
const (
AttributeKeyItemId = "item_id"
AttributeKeyCreator = "creator"
AttributeKeyName = "name"
)
```

2. 이벤트 발행

```
// 상세한 이벤트 정보 제공
ctx.EventManager().EmitEvent(
sdk.NewEvent(
types.EventTypeCreateItem,
sdk.NewAttribute(types.AttributeKeyItemId, fmt.Sprintf("%d", id)),
```

```
sdk.NewAttribute(types.AttributeKeyCreator, req.Creator),
sdk.NewAttribute(types.AttributeKeyName, req.Name),
sdk.NewAttribute(sdk.AttributeKeyModule, types.ModuleName),
),
)
```

5.6.4 파라미터 관리

1. 파라미터 구조체 설계

```
type Params struct {
MaxItemNameLength uint32  `protobuf:"varint,1,opt,name=max_item_name_length,json
MaxItemDescriptionLength uint32  `protobuf:"varint,2,opt,name=max_item_descripti
CreationFee string  `protobuf:"bytes,3,opt,name=creation_fee,json=creationFee,pr
}
```

2. 파라미터 검증

```
func (p Params) Validate() error {
if p.MaxItemNameLength == 0 {
return fmt.Errorf("max item name length must be positive")
}
if p.MaxItemDescriptionLength == 0 {
return fmt.Errorf("max item description length must be positive")
}
```

```go
if _, err := sdk.ParseCoinNormalized(p.CreationFee); err != nil {
return fmt.Errorf("invalid creation fee: %w", err)
}
return  nil
}
```

6. 테스트 작성 가이드

6.1 단위 테스트 구조

```go
// x/mymodule/keeper/keeper_test.go
package keeper_test

import (
"testing"
"github.com/stretchr/testify/require"
"mychain/x/mymodule/keeper"
"mychain/x/mymodule/types"
)

type fixture struct {
ctx context.Context
keeper keeper.Keeper
```

```go
}


func initFixture(t *testing.T) *fixture {
// 테스트 환경 설정
encCfg := moduletestutil.MakeTestEncodingConfig()
key := storetypes.NewKVStoreKey(types.StoreKey)
storeService := runtime.NewKVStoreService(key)
testCtx := testutil.DefaultContextWithDB(t, key, storetypes.NewTransientStoreKey
ctx := testCtx.Ctx
// Keeper 생성
k := keeper.NewKeeper(
storeService,
encCfg.Codec,
address.NewBech32Codec("cosmos"),
authtypes.NewModuleAddress("gov"),
)
return &fixture{
ctx: ctx,
keeper: k,
}
}


func TestCreateItem(t *testing.T) {
```

```
f := initFixture(t)

// 테스트 데이터

item := types.Item{

Name: "Test Item",

Description: "Test Description",

Creator: "cosmos1...",

}

// 아이템 생성

id, err := f.keeper.NextId.Next(f.ctx)

require.NoError(t, err)

item.Id = id

err = f.keeper.Items.Set(f.ctx, id, item)

require.NoError(t, err)

// 검증

savedItem, err := f.keeper.Items.Get(f.ctx, id)

require.NoError(t, err)

require.Equal(t, item, savedItem)

}
```

6.2 메시지 핸들러 테스트

```
func TestMsgCreateItem(t *testing.T) {
```

```
f := initFixture(t)

msgServer := keeper.NewMsgServerImpl(f.keeper)

testCases := []struct {

name string

msg *types.MsgCreateItem

expErr bool

errMsg string

}{

{

name: "valid message",

msg: &types.MsgCreateItem{

Name: "Test Item",

Description: "Test Description",

Creator: "cosmos1...",

},

expErr: false,

},

{

name: "empty name",

msg: &types.MsgCreateItem{

Name: "",

Description: "Test Description",

Creator: "cosmos1...",
```

```
},
expErr: true,
errMsg: "invalid name",
},
}
for _, tc := range testCases {
t.Run(tc.name, func(t *testing.T) {
resp, err := msgServer.CreateItem(f.ctx, tc.msg)
if tc.expErr {
require.Error(t, err)
require.Contains(t, err.Error(), tc.errMsg)
require.Nil(t, resp)
} else {
require.NoError(t, err)
require.NotNil(t, resp)
require.Greater(t, resp.Id, uint64(0))
}
})
}
}
```

6.3 통합 테스트

```go
// integration_test.go
func TestFullWorkflow(t *testing.T) {
// 전체 앱 초기화
app := simapp.New(log.NewNopLogger(), dbm.NewMemDB(), nil, true, simtestutil.New
ctx := app.BaseApp.NewContext(false, tmproto.Header{})
// 계정 설정
creator := sdk.AccAddress("creator")
app.AccountKeeper.SetAccount(ctx, app.AccountKeeper.NewAccountWithAddress(ctx, c
// 토큰 발행
coins := sdk.NewCoins(sdk.NewCoin("stake", sdk.NewInt(1000000)))
require.NoError(t, app.BankKeeper.MintCoins(ctx, minttypes.ModuleName, coins))
require.NoError(t, app.BankKeeper.SendCoinsFromModuleToAccount(ctx, minttypes.Mo
// 메시지 실행
msg := &types.MsgCreateItem{
Name: "Integration Test Item",
Description: "Created in integration test",
Creator: creator.String(),
}
msgServer := keeper.NewMsgServerImpl(app.MyModuleKeeper)
resp, err := msgServer.CreateItem(ctx, msg)
require.NoError(t, err)
require.NotNil(t, resp)
```

```
// 상태 검증
item, err := app.MyModuleKeeper.Items.Get(ctx, resp.Id)
require.NoError(t, err)
require.Equal(t, msg.Name, item.Name)
}
```

6.4 시뮬레이션 테스트

```
// simulation.go
func SimulateMsgCreateItem(
ak types.AccountKeeper,
bk types.BankKeeper,
k keeper.Keeper,
) simtypes.Operation {
return  func(r *rand.Rand, app *baseapp.BaseApp, ctx sdk.Context, accs []simtype
) (simtypes.OperationMsg, []simtypes.FutureOperation, error) {
// 랜덤 계정 선택
simAccount, _ := simtypes.RandomAcc(r, accs)
// 랜덤 데이터 생성
name := simtypes.RandStringOfLength(r, 10)
description := simtypes.RandStringOfLength(r, 50)
msg := &types.MsgCreateItem{
Name: name,
```

```
Description: description,

Creator: simAccount.Address.String(),

}

// 메시지 실행

txCtx := simulation.OperationInput{

R: r,

App: app,

TxGen: simappparams.MakeTestEncodingConfig().TxConfig,

Cdc: nil,

Msg: msg,

MsgType: msg.Type(),

Context: ctx,

SimAccount: simAccount,

AccountKeeper: ak,

Bankkeeper: bk,

ModuleName: types.ModuleName,

}

return simulation.GenAndDeliverTxWithRandFees(txCtx)

}

}
```

7. IBC 통합 가이드

7.1 IBC 애플리케이션 개발

IBC 애플리케이션을 개발하려면 `porttypes.IBCModule` 인터페이스를 구현해야 합니다.

```go
// x/mymodule/ibc_module.go
package mymodule

import (
sdk "github.com/cosmos/cosmos-sdk/types"
channeltypes "github.com/cosmos/ibc-go/v10/modules/core/04-channel/types"
porttypes "github.com/cosmos/ibc-go/v10/modules/core/05-port/types"
"mychain/x/mymodule/keeper"
"mychain/x/mymodule/types"
)

type IBCModule struct {
keeper keeper.Keeper
}

func NewIBCModule(k keeper.Keeper) IBCModule {
return IBCModule{
keeper: k,
}
}
```

```go
// 채널 핸드셰이크 콜백들
func (im IBCModule) OnChanOpenInit(
ctx sdk.Context,
order channeltypes.Order,
connectionHops []string,
portID string,
channelID string,
chanCap *capabilitytypes.Capability,
counterparty channeltypes.Counterparty,
version string,
) (string, error) {
// 채널 초기화 로직
if version != types.Version {
return  "", types.ErrInvalidVersion
}
// 채널 capability 저장
if err := im.keeper.ClaimCapability(ctx, chanCap, host.ChannelCapabilityPath(por
return  "", err
}
return version, nil
}
```

```go
func (im IBCModule) OnChanOpenTry(
ctx sdk.Context,
order channeltypes.Order,
connectionHops []string,
portID,
channelID string,
chanCap *capabilitytypes.Capability,
counterparty channeltypes.Counterparty,
counterpartyVersion string,
) (string, error) {
// 상대방 채널 연결 시도 처리
return types.Version, nil
}


// 패킷 처리 콜백들
func (im IBCModule) OnRecvPacket(
ctx sdk.Context,
packet channeltypes.Packet,
relayer sdk.AccAddress,
) ibcexported.Acknowledgement {
// 패킷 수신 처리
var data types.MyPacketData
if err := types.ModuleCdc.UnmarshalJSON(packet.GetData(), &data); err != nil {
```

```go
return channeltypes.NewErrorAcknowledgement(err)
}
// 비즈니스 로직 실행
if err := im.keeper.ProcessPacket(ctx, data); err != nil {
return channeltypes.NewErrorAcknowledgement(err)
}
return channeltypes.NewResultAcknowledgement([]byte("success"))
}


func (im IBCModule) OnAcknowledgementPacket(
ctx sdk.Context,
packet channeltypes.Packet,
acknowledgement []byte,
relayer sdk.AccAddress,
) error {
// 확인 패킷 처리
var ack channeltypes.Acknowledgement
if err := types.ModuleCdc.UnmarshalJSON(acknowledgement, &ack); err != nil {
return err
}
switch resp := ack.Response.(type) {
case *channeltypes.Acknowledgement_Result:
// 성공 처리
```

```
return im.keeper.OnPacketSuccess(ctx, packet)
case *channeltypes.Acknowledgement_Error:
// 실패 처리
return im.keeper.OnPacketFailure(ctx, packet, resp.Error)
}
return  nil
}
```

7.2 IBC 패킷 전송

```
// Keeper에서 IBC 패킷 전송
func (k Keeper) SendIBCPacket(ctx sdk.Context, channelID string, data types.MyPa
// 패킷 데이터 직렬화
packetData, err := types.ModuleCdc.MarshalJSON(&data)
if err != nil {
return err
}
// 타임아웃 설정
timeoutHeight := clienttypes.NewHeight(0, 0) // 높이 기반 타임아웃 없음
timeoutTimestamp := uint64(ctx.BlockTime().Add(time.Hour).UnixNano()) // 1시간 후
// 패킷 전송
_, err = k.channelKeeper.SendPacket(
ctx,
```

```
capabilitytypes.NewCapability(1), // 채널 capability
types.PortID,
channelID,
timeoutHeight,
timeoutTimestamp,
packetData,
)
return err
}
```

7.3 IBC 모듈 등록

```
// app/ibc.go에서 IBC 모듈 등록
func (app *App) registerIBCModules() error {
// 커스텀 IBC 모듈 생성
myIBCModule := mymodule.NewIBCModule(app.MyModuleKeeper)
// 라우터에 등록
ibcRouter.AddRoute(types.ModuleName, myIBCModule)
return  nil
}
```

8. 성능 최적화

8.1 저장소 최적화

```
// 배치 처리를 통한 성능 향상
func (k Keeper) BatchUpdateItems(ctx sdk.Context, items []types.Item) error {
// 트랜잭션 내에서 배치 처리
return k.storeService.OpenKVStore(ctx).Batch(func(batch storetypes.Batch) error
for _, item := range items {
if err := k.Items.Set(ctx, item.Id, item); err != nil {
return err
}
}
return  nil
})
}

// 인덱스를 활용한 효율적인 쿼리
func (k Keeper) GetItemsByCreator(ctx sdk.Context, creator string) ([]types.Item
var items []types.Item
// 인덱스를 사용한 범위 쿼리
err := k.ItemsByCreator.Walk(ctx, collections.NewPrefixedPairRange[string, uint6
func(key collections.Pair[string, uint64], value collections.NoValue) (bool, err
item, err := k.Items.Get(ctx, key.K2())
if err != nil {
```

```go
return  true, err
}
items = append(items, item)
return  false, nil
})
return items, err
}
```

8.2 메모리 최적화

```go
// 메모리 풀 사용
var itemPool = sync.Pool{
New: func() interface{} {
return &types.Item{}
},
}

func (k Keeper) ProcessItem(ctx sdk.Context, data []byte) error {
// 풀에서 객체 가져오기
item := itemPool.Get().(*types.Item)
defer itemPool.Put(item)
// 객체 재사용
if err := k.cdc.Unmarshal(data, item); err != nil {
```

```
return err
}
// 처리 로직
return k.Items.Set(ctx, item.Id, *item)
}

// 스트리밍을 통한 대용량 데이터 처리
func (k Keeper) StreamItems(ctx sdk.Context, callback func(types.Item) error) er
return k.Items.Walk(ctx, nil, func(key uint64, value types.Item) (bool, error) {
if err := callback(value); err != nil {
return  true, err
}
return  false, nil
})
}
```

8.3 캐싱 전략

```
// LRU 캐시 구현
type CachedKeeper struct {
Keeper
cache *lru.Cache
}
```

```go
func NewCachedKeeper(keeper Keeper, cacheSize int) *CachedKeeper {
cache, _ := lru.New(cacheSize)
return &CachedKeeper{
Keeper: keeper,
cache: cache,
}
}

func (k *CachedKeeper) GetItem(ctx sdk.Context, id uint64) (types.Item, error) {
// 캐시에서 먼저 확인
if cached, ok := k.cache.Get(id); ok {
return cached.(types.Item), nil
}
// 캐시 미스 시 저장소에서 조회
item, err := k.Keeper.Items.Get(ctx, id)
if err != nil {
return types.Item{}, err
}
// 캐시에 저장
k.cache.Add(id, item)
return item, nil
```

```
}
```

9. 보안 모범 사례

9.1 입력 검증

```
// 철저한 입력 검증
func (k Keeper) ValidateCreateItemMsg(msg *types.MsgCreateItem) error {
// 필수 필드 검증
if strings.TrimSpace(msg.Name) == "" {
return types.ErrInvalidName
}
// 길이 제한
if len(msg.Name) > types.MaxNameLength {
return types.ErrNameTooLong
}
if len(msg.Description) > types.MaxDescriptionLength {
return types.ErrDescriptionTooLong
}
// 주소 형식 검증
if _, err := k.addressCodec.StringToBytes(msg.Creator); err != nil {
return types.ErrInvalidAddress
}
```

```go
// 특수 문자 검증
if !isValidName(msg.Name) {
return types.ErrInvalidCharacters
}
return  nil
}


func isValidName(name string) bool {
// 알파벳, 숫자, 하이픈, 언더스코어만 허용
matched, _ := regexp.MatchString(`^[a-zA-Z0-9_-]+$`, name)
return matched
}
```

9.2 권한 관리

```go
// 세밀한 권한 제어
func (k Keeper) CheckPermission(ctx sdk.Context, action string, actor sdk.AccAdd
// 관리자 권한 확인
if k.IsAdmin(ctx, actor) {
return  nil
}
// 리소스 소유자 확인
if k.IsOwner(ctx, actor, resource) {
```

```go
return  nil
}
// 특정 권한 확인
if k.HasPermission(ctx, actor, action, resource) {
return  nil
}
return types.ErrUnauthorized
}


// 역할 기반 접근 제어 (RBAC)
type Role string


const (
RoleAdmin Role = "admin"
RoleModerator Role = "moderator"
RoleUser Role = "user"
)


func (k Keeper) HasRole(ctx sdk.Context, addr sdk.AccAddress, role Role) bool {
userRole, err := k.UserRoles.Get(ctx, addr)
if err != nil {
return  false
}
```

```
return userRole >= role

}
```

9.3 레이트 리미팅

```
// 사용자별 레이트 리미팅

func (k Keeper) CheckRateLimit(ctx sdk.Context, addr sdk.AccAddress, action stri

key := fmt.Sprintf("%s:%s", addr.String(), action)

// 현재 시간 윈도우

window := ctx.BlockTime().Truncate(time.Minute)

// 현재 카운트 조회

count, err := k.RateLimits.Get(ctx, key)

if err != nil && !errors.Is(err, collections.ErrNotFound) {

return err

}

// 제한 확인

limit := k.GetRateLimit(action)

if count >= limit {

return types.ErrRateLimitExceeded

}

// 카운트 증가

return k.RateLimits.Set(ctx, key, count+1)
```

```
}
```

9.4 암호화 및 해싱

```
// 민감한 데이터 해싱
func (k Keeper) HashSensitiveData(data string) []byte {
hasher := sha256.New()
hasher.Write([]byte(data))
hasher.Write(k.GetSalt()) // 솔트 추가
return hasher.Sum(nil)
}

// 서명 검증
func (k Keeper) VerifySignature(ctx sdk.Context, msg []byte, sig []byte, pubKey
if !pubKey.VerifySignature(msg, sig) {
return types.ErrInvalidSignature
}
// 재사용 공격 방지를 위한 nonce 확인
nonce := k.GetNonce(ctx, pubKey.Address())
expectedNonce := binary.BigEndian.Uint64(msg[:8])
if nonce != expectedNonce {
return types.ErrInvalidNonce
}
```

```
// nonce 증가
k.SetNonce(ctx, pubKey.Address(), nonce+1)
return nil
}
```

10. 배포 및 운영

10.1 네트워크 설정

```
# 제네시스 파일 생성
marsd init mynode --chain-id mars-1

# 검증자 키 생성
marsd keys add validator

# 제네시스 계정 추가
marsd genesis add-genesis-account validator 1000000000stake

# 제네시스 트랜잭션 생성
marsd genesis gentx validator 1000000stake --chain-id mars-1

# 제네시스 트랜잭션 수집
marsd genesis collect-gentxs
```

```
# 제네시스 파일 검증
marsd genesis  validate-genesis
```

10.2 설정 파일 최적화

```
# config/config.toml
[consensus]
timeout_propose = "3s"
timeout_propose_delta = "500ms"
timeout_prevote = "1s"
timeout_prevote_delta = "500ms"
timeout_precommit = "1s"
timeout_precommit_delta = "500ms"
timeout_commit = "5s"

[mempool]
size = 5000
max_txs_bytes = 1073741824
cache_size = 10000

[p2p]
max_num_inbound_peers = 40
```

```
max_num_outbound_peers = 10

persistent_peers = "node1@ip1:26656,node2@ip2:26656"



# config/app.toml

[api]

enable = true

swagger = true

address = "tcp://0.0.0.0:1317"



[grpc]

enable = true

address = "0.0.0.0:9090"



[state-sync]

snapshot-interval = 1000

snapshot-keep-recent = 2
```

10.3 모니터링 설정

```yaml
# docker-compose.yml

version: '3.8'

services:

mars-node:
```

```yaml
image: mars:latest
ports:
- "26656:26656"
- "26657:26657"
- "1317:1317"
- "9090:9090"
volumes:
- ./data:/root/.mars
environment:
- MARS_CHAIN_ID=mars-1
prometheus:
image: prom/prometheus
ports:
- "9091:9090"
volumes:
- ./prometheus.yml:/etc/prometheus/prometheus.yml
grafana:
image: grafana/grafana
ports:
- "3000:3000"
environment:
- GF_SECURITY_ADMIN_PASSWORD=admin
```

```yaml
# prometheus.yml
global:
scrape_interval: 15s

scrape_configs:
- job_name: 'mars-node'
static_configs:
- targets: ['mars-node:26660']
```

10.4 백업 및 복구

```bash
#!/bin/bash
# backup.sh

# 데이터 디렉토리 백업
tar -czf  mars-backup-$(date +%Y%m%d).tar.gz  ~/.mars/data

# 설정 파일 백업
cp ~/.mars/config/genesis.json  ./genesis-backup.json
cp ~/.mars/config/config.toml  ./config-backup.toml
cp ~/.mars/config/app.toml  ./app-backup.toml

# S3 업로드 (선택사항)
```

```
aws s3 cp mars-backup-$(date +%Y%m%d).tar.gz s3://mars-backups/
```

11. 문제 해결

11.1 일반적인 문제들

빌드 실패

```
# 의존성 문제인 경우
make clean
make tidy
make install
```

테스트 실패

```
# 캐시 문제인 경우
make clean
make test-unit
```

프로토콜 버퍼 생성 실패

```
# proto 의존성 확인
buf mod update
# 재생성 시도
```

```
make proto-gen
```

11.2 성능 문제 해결

느린 쿼리 최적화

```
// 인덱스 추가
ItemsByStatus: collections.NewIndexedMap(
sb, types.ItemsByStatusKey, "items_by_status",
collections.Uint64Key,
codec.CollValue[types.Item](cdc),
types.NewItemsStatusIndex(sb),
)

// 페이지네이션 구현
func (k Keeper) ListItemsPaginated(ctx sdk.Context, req *types.QueryListItemsReq
var items []types.Item
pageRes, err := query.CollectionPaginate(
ctx,
k.Items,
req.Pagination,
func(key uint64, value types.Item) (types.Item, error) {
return value, nil
},
```

```
)
if err != nil {
return  nil, err
}
return &types.QueryListItemsResponse{
Items: items,
Pagination: pageRes,
}, nil
}
```

메모리 사용량 최적화

```
// 대용량 데이터 스트리밍 처리
func (k Keeper) ProcessLargeDataset(ctx sdk.Context, callback func(types.Item) e
return k.Items.Walk(ctx, nil, func(key uint64, value types.Item) (bool, error) {
if err := callback(value); err != nil {
return  true, err
}
return  false, nil
})
}
```

11.3 보안 문제 해결

권한 에스컬레이션 방지

```
func (k Keeper) ValidateAuthority(authority string) error {
expectedAuthority := k.GetAuthority()
if authority != string(expectedAuthority) {
return sdkerrors.Wrapf(
govtypes.ErrInvalidSigner,
"invalid authority; expected %s, got %s",
expectedAuthority, authority,
)
}
return  nil
}
```

입력 검증 강화

```
func (k Keeper) ValidateInput(input string) error {
// 길이 제한
if len(input) > MaxInputLength {
return types.ErrInputTooLong
}
// 특수 문자 필터링
if containsSpecialChars(input) {
return types.ErrInvalidCharacters
```

```
}

// SQL 인젝션 방지

if containsSQLKeywords(input) {

return types.ErrSuspiciousInput

}

return  nil

}
```

결론

이 개발자 가이드는 Mars 블록체인 프로젝트의 전체적인 구조와 개발 방법론을 포괄적으로 다루고 있습니다. Cosmos SDK의 최신 패턴과 모범 사례를 따라 안정적이고 확장 가능한 블록체인 애플리케이션을 개발할 수 있도록 구성되었습니다.

핵심 포인트 요약

1. **모듈화된 아키텍처**: 각 기능을 독립적인 모듈로 구성하여 유지보수성 향상
2. **Collections 기반 상태 관리**: 타입 안전한 저장소 시스템으로 개발 효율성 증대
3. **의존성 주입**: 자동화된 의존성 해결로 코드 복잡성 감소
4. **포괄적인 테스트**: 단위 테스트부터 시뮬레이션 테스트까지 다양한 검증 방법
5. **보안 우선 설계**: 입력 검증, 권한 관리, 암호화 등 보안 모범 사례 적용

이 가이드를 통해 개발자들은 Mars 블록체인 생태계에 기여할 수 있는 고품질의 모듈과 애플리케이션을 개발할 수 있을 것입니다.